JavaScript Logic Building Practice Sheet

Oct 12, 2026 · @Upendra

Level 1 – Basic

1. Even or Odd

Write a function that takes a number and returns "Even" or "Odd".

```
evenOrOdd(7)   // "Odd"
evenOrOdd(12)  // "Even"
```

2. Largest of Three

Write a function that returns the largest of three numbers without using `Math.max`.

```
largest(4, 19, 7)   // 19
largest(-2, -8, -1) // -1
```

3. Sum of Digits

Write a function that returns the sum of all digits of a positive integer.

```
sumDigits(4512)  // 12
sumDigits(9)     // 9
```

4. Reverse a String

Reverse a string without using the built-in `reverse()` method.

```
reverseStr("hello")      // "olleh"
reverseStr("JavaScript") // "tpircSavaJ"
```

5. Count Vowels

Count the number of vowels (a, e, i, o, u) in a string, ignoring case.

```
countVowels("Programming") // 3
countVowels("AEIOU xyz")   // 5
```

6. Factorial

Return the factorial of a non-negative integer n.

```
factorial(5) // 120
factorial(0) // 1
```

7. Prime Check

Return `true` if a number is prime, otherwise `false`.

```
isPrime(17) // true
isPrime(21) // false
isPrime(1)  // false
```

8. FizzBuzz

For numbers 1 to n, return an array where multiples of 3 become "Fizz", multiples of 5 become "Buzz", multiples of both become "FizzBuzz", and the rest stay as numbers.

```
fizzBuzz(15)
// [1, 2, "Fizz", 4, "Buzz", "Fizz", 7, 8, "Fizz", "Buzz", 11, "Fizz", 13,
14, "FizzBuzz"]
```

9. Find the Maximum in an Array

Return the largest number in an array using a loop (no `Math.max` or `sort`).

```
findMax([3, 41, 7, 19, 2]) // 41
findMax([-5, -1, -9])      // -1
```

10. Palindrome Check

Return `true` if a string reads the same forwards and backwards, ignoring case and spaces.

```
isPalindrome("Racecar")          // true
isPalindrome("never odd or even") // true
isPalindrome("hello")            // false
```

11. Multiplication Table

Return an array with the multiplication table of a number from 1 to 10.

```
table(3) // [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
```

12. Count Words

Count the number of words in a sentence. Extra spaces between words should not be counted as words.

```
countWords("I love  JavaScript") // 3
countWords("  hello  ")          // 1
```

Level 2 – Intermediate

13. Remove Duplicates

Return a new array with duplicates removed, keeping the first occurrence order. Solve it once without `Set`.

```
removeDuplicates([1, 3, 3, 7, 1, 9]) // [1, 3, 7, 9]
```

14. Character Frequency

Return an object counting how many times each character appears in a string.

```
charCount("banana") // { b: 1, a: 3, n: 2 }
```

15. Anagram Check

Return `true` if two strings are anagrams of each other (same letters, same counts), ignoring case.

```
isAnagram("Listen", "Silent") // true
isAnagram("hello", "world")   // false
```

16. Fibonacci Series

Return an array with the first n numbers of the Fibonacci sequence.

```
fibonacci(8) // [0, 1, 1, 2, 3, 5, 8, 13]
```

17. Second Largest Number

Find the second largest distinct number in an array in a single pass, without sorting.

```
secondLargest([12, 35, 1, 10, 34, 35]) // 34
secondLargest([5, 5, 5])               // null
```

18. Capitalize Each Word

Capitalize the first letter of every word in a sentence.

```
capitalize("learn javascript every day") // "Learn Javascript Every Day"
```

19. Missing Number

An array contains n−1 distinct numbers from 1 to n. Find the missing number.

```
findMissing([1, 2, 4, 5, 6]) // 3
findMissing([2, 3, 1, 5])    // 4
```

20. Chunk an Array

Split an array into smaller arrays of a given size.

```
chunk([1, 2, 3, 4, 5, 6, 7], 3) // [[1, 2, 3], [4, 5, 6], [7]]
```

21. Rotate an Array

Rotate an array to the right by k steps.

```
rotate([1, 2, 3, 4, 5], 2) // [4, 5, 1, 2, 3]
rotate([1, 2, 3], 4)       // [3, 1, 2]
```

22. Two Sum

Given an array and a target, return the indices of the two numbers that add up to the target.

```
twoSum([2, 7, 11, 15], 9) // [0, 1]
twoSum([3, 2, 4], 6)      // [1, 2]
```

Level 3 – Advanced

23. Flatten a Nested Array

Flatten an array of any depth into a single-level array without using `flat()`.

```
flatten([1, [2, [3, [4, 5]]], 6]) // [1, 2, 3, 4, 5, 6]
```

24. Longest Substring Without Repeating Characters

Return the length of the longest substring that has no repeating characters.

```
longestUnique("abcabcbb") // 3  ("abc")
longestUnique("pwwkew")   // 3  ("wke")
```

25. Group Anagrams

Group words that are anagrams of each other.

```
groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
// [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

26. Balanced Brackets

Return `true` if every bracket in the string is closed in the correct order. Brackets are `()`, `[]` and `{}`.

```
isBalanced("{[()]}") // true
isBalanced("([)]")   // false
isBalanced("((")     // false
```

27. Deep Clone an Object

Write a function that deep-copies an object containing nested objects and arrays, without `structuredClone` or `JSON` methods. Changing the copy must not affect the original.

```
const original = { a: 1, b: { c: [1, 2] } };
const copy = deepClone(original);
copy.b.c.push(3);
original.b.c // [1, 2]
copy.b.c     // [1, 2, 3]
```

28. Debounce

Implement `debounce(fn, delay)`: the returned function runs `fn` only after it has not been called for `delay` milliseconds.

```
const log = debounce(() => console.log("Searching..."), 300);
log(); log(); log();
// "Searching..." is printed once, 300 ms after the last call
```

29. All Permutations of a String

Return every possible arrangement of the characters in a string.

```
permutations("abc")
// ["abc", "acb", "bac", "bca", "cab", "cba"]
```

30. Implement Your Own Promise.all

Write `promiseAll(promises)` that resolves with an array of results in the original order once all promises resolve, and rejects as soon as any one rejects.

```
const p1 = Promise.resolve(1);
const p2 = new Promise(res => setTimeout(() => res(2), 100));
const p3 = 3;

promiseAll([p1, p2, p3]).then(console.log); // [1, 2, 3]
promiseAll([p1, Promise.reject("Error!")]).catch(console.log); // "Error!"
```